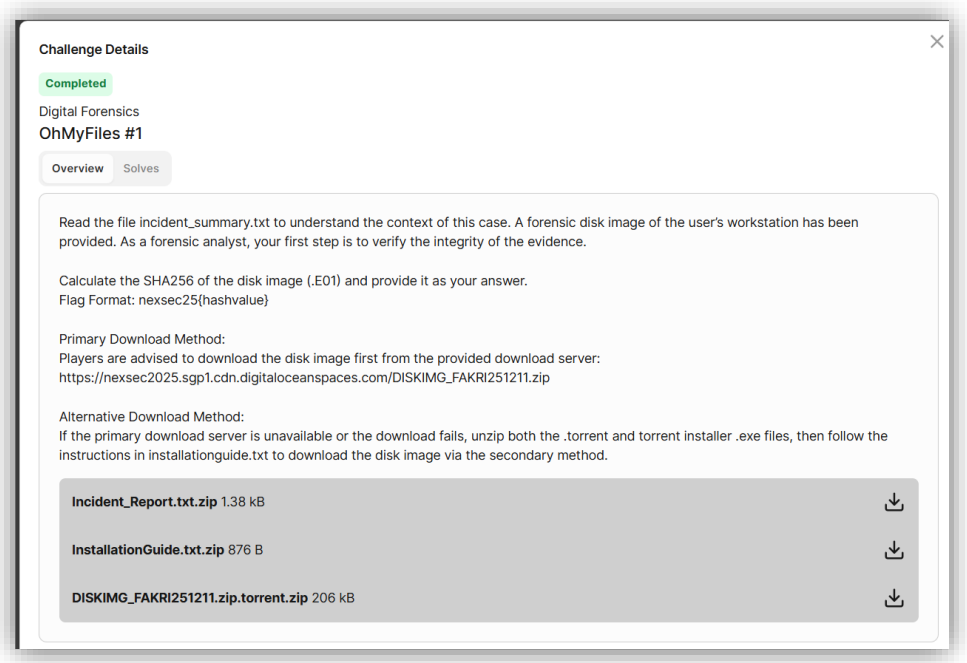
OhMyFiles #1

Challenge Details



Challenge Details

Completed

Digital Forensics

OhMyFiles #1

Overview Solves

Read the file incident_summary.txt to understand the context of this case. A forensic disk image of the user's workstation has been provided. As a forensic analyst, your first step is to verify the integrity of the evidence.

Calculate the SHA256 of the disk image (.E01) and provide it as your answer.
Flag Format: nexsec25{hashvalue}

Primary Download Method:
Players are advised to download the disk image first from the provided download server:
https://nexsec2025.sgp1.cdn.digitaloceanspaces.com/DISKIMG_FAKRI251211.zip

Alternative Download Method:
If the primary download server is unavailable or the download fails, unzip both the .torrent and torrent installer .exe files, then follow the instructions in installationguide.txt to download the disk image via the secondary method.

Incident_Report.txt.zip 1.38 kB

InstallationGuide.txt.zip 876 B

DISKIMG_FAKRI251211.zip.torrent.zip 206 kB

Main Task

Download the forensic disk image and obtain its SHA-256 hash value.

Initial Findings

Before starting the analysis, I reviewed the provided materials.

Installation Guide Summary

```
┌──(kali㉿kali)-[~/Desktop/ohmyfiles1]
└─$ cat InstallationGuide.txt
═══════════════════════════════════════
INSTRUCTIONS TO DOWNLOAD DISK IMAGE
USING .TORRENT FILE
═══════════════════════════════════════

Step 1: Install a torrent client
────────────────────────────────
If you don't have a torrent client, install it from the attachmnt (qbittorrent_5.1.4_x64_s
etup.exe)

Step 2: Open the torrent client
───────────────────────────────
Launch the torrent client after installation.

Step 3: Add the .torrent file
─────────────────────────────
1. In the client, choose File → Add Torrent / Open Torrent.
2. Browse and select the .torrent file you received.  (DISKIMG_FAKRI251211.zip.torrent)
3. Click OK / Add to start downloading.

Step 4: Choose download location
────────────────────────────────
Select a folder on your computer where you want the .zip file to be saved.
Make sure there is enough space (~10.7 GB).

Step 5: Wait for download to complete
─────────────────────────────────────
- The torrent client will download the file in pieces from the seeder(s).
- Download speed depends on the number of seeders.
- Once finished, you will have the full .zip disk image ready to use.

Important Notes
───────────────
- The seeder (person who created the torrent) must keep their client open while others are
 downloading.
- If there are no active seeders, the download cannot complete.
- After downloading, you can extract or use the .zip as needed.

═══════════════════════════════════════
```

The disk image is large, so downloading it through a torrent client is more reliable than using a browser. A normal browser download may fail if the internet connection is interrupted, the speed may be slow, and the process may need to be restarted. Therefore, using a torrent client is the better option.

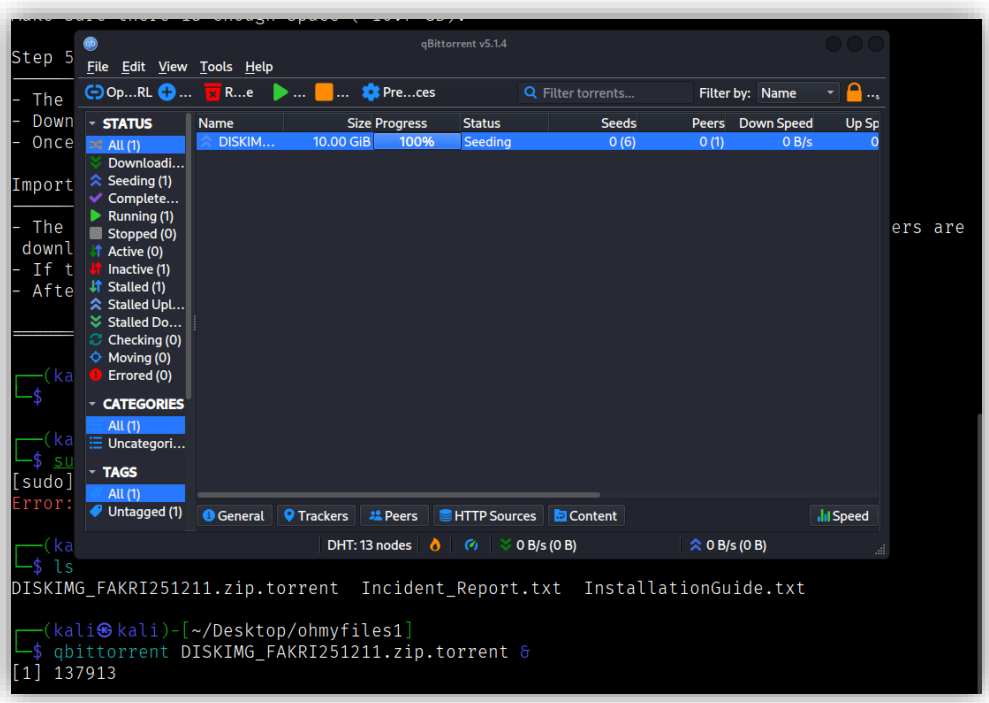
Incident Report Summary

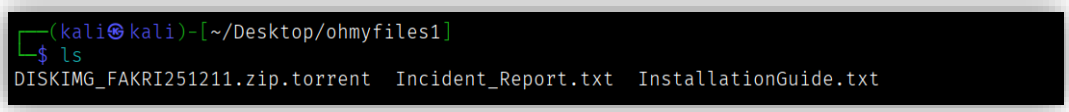
```
┌──(kali㉿kali)-[~/Desktop/ohmyfiles1]
└─$ cat Incident_Report.txt
═══════════════════════════════════════════════════
              INCIDENT SUMMARY REPORT
═══════════════════════════════════════════════════

User: En. Fakhri
Reported Issue: Encrypted and unreadable work document

___________________________________________________
                DESCRIPTION OF EVENT
___________________________________________________
En. Fakhri contacted the IT Security team with an urgent issue.
He was preparing to send an important document named:

    BigClient_Proposal_2025.docx

However, he discovered that the file had been encrypted and was no longer readable.

___________________________________________________
                 USER ACTIVITIES
___________________________________________________
Before the incident occurred, Fakhri reported that he:

1. Downloaded a compressed archive (.rar file) earlier that day.
2. The archive contained a resume template.
3. He extracted the contents.
4. He then deleted the .rar file after extraction.

___________________________________________________
             FORENSIC ACTION TAKEN
___________________________________________________
The IT Security team created a forensic image of Fakhri's workstation
for detailed investigation and recovery of encrypted files.

Disk Image:
https://mega.nz/file/itJmzS5Y#eW62vkwrS4DMWq8o_r_IB-VuVjvuBqhUhAk2iTIIs04

___________________________________________________
                    END OF FILE
═══════════════════════════════════════════════════
```

The report stated that En. Fakhri's files were encrypted by ransomware. One of the important affected documents was **BigClient_Proposal_2025.docx**. According to the report, Fakhri had downloaded a compressed archive, extracted it, obtained a resume template, and later deleted the archive. After the incident, the IT security team created a forensic image of his workstation.

Step 1: Download the Disk Image

First, I installed a torrent client in Kali Linux to download the disk image.





Command Used:

1. sudo apt install qbittorrent
2. qbittorent DISKIMG_FAKRI251211.zip.torrent &

Step 2: Calculate the SHA-256 Hash

After the download was completed, I calculated the SHA-256 hash of the forensic image file.

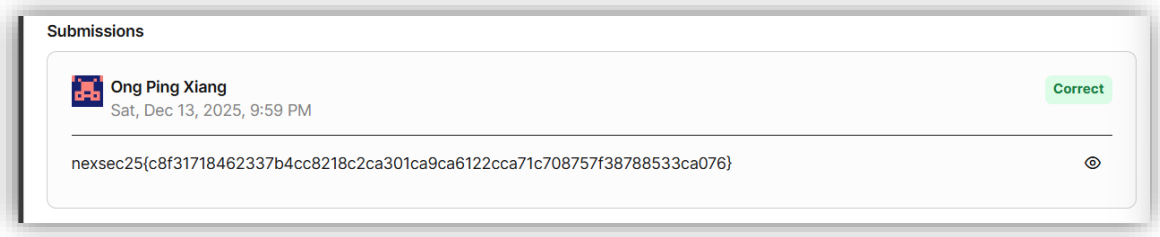
Command used:

1. sha256sum FAKHRIWORKSTATION_20251211.E01

Final Answer

The SHA-256 value of the disk image is:

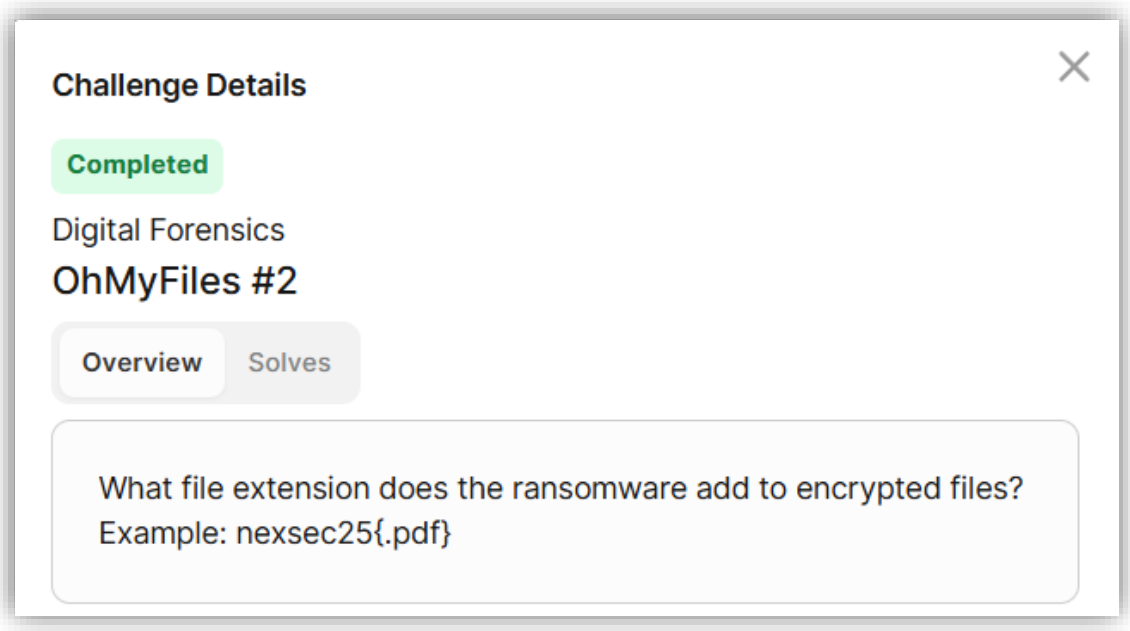
c8f31718462337b4cc8218c2ca301ca9ca6122cca71c708757f38788533ca076



Flag: **nexsec25{c8f31718462337b4cc8218c2ca301ca9ca6122cca71c708757f38788533ca076}**

OhMyFiles #2

Challenge Details



Main Task

Determine the file extension used by the ransomware on the victim's encrypted files.

Initial Approach

To answer this question, I first needed to identify the main partition in the forensic disk image. Since user files are usually stored in the Windows system partition, locating the correct partition would help narrow down the search.

Step 1: Identify the Main Partition

```
kali@kali: ~/Downloads/DISKIMG_FAKRI251211
Session  Actions  Edit  View  Help
└─$ mmls FAKHRIWORKSTATION_20251211.E01
GUID Partition Table (EFI)
Offset Sector: 0
Units are in 512-byte sectors

      Slot      Start        End          Length       Description
000:  Meta      0000000000   0000000000   0000000001   Safety Table
001:  -------   0000000000   0000002047   0000002048   Unallocated
002:  Meta      0000000001   0000000001   0000000001   GPT Header
003:  Meta      0000000002   0000000033   0000000032   Partition Table
004:  000       0000002048   0000411647   0000409600   EFI system partition
005:  001       0000411648   0000673791   0000262144   Microsoft reserved partition
006:  002       0000673792   0104855551   0104181760   Basic data partition
007:  -------   0104855552   0104857599   0000002048   Unallocated
```

I used the mmls command to inspect the partition layout of the disk image. This command shows how the disk is divided, including where each partition starts and ends.

Command used:

1. mmls FAKHRIWORKSTATION_20251211.E01

Based on the output, several partitions were listed:

- **Slot 004** – EFI system partition
  This partition is used for booting and is not relevant to user files.
- **Slot 005** – Microsoft reserved partition
  This is a small system partition.
- **Slot 006** – Basic data partition
  This is the main Windows partition and is the largest partition in the image. It begins at sector **673792** and contains the user data, including the Users folder.

Therefore, the investigation was continued using offset 673792.

Step 2: Search the User Directory

After identifying the correct partition, I searched through the victim's user directory to look for signs of ransomware activity.

```
┌──(kali㉿kali)-[~/Downloads/DISKIMG_FAKRI251211]
└─$ fls -r -p -o 673792 FAKHRIWORKSTATION_20251211.E01 | grep "Users/Fakhri/"
r/r 105175-128-1:       Users/Fakhri/NTUSER.DAT{53b39e88-18c4-11ea-a811-000d3aa4692b}.TM.b
lf
r/r 119327-128-1:       Users/Fakhri/.python_history
d/d 105594-144-1:       Users/Fakhri/3D Objects
r/r 105595-128-1:       Users/Fakhri/3D Objects/desktop.ini
d/d 105086-144-1:       Users/Fakhri/AppData
d/d 105105-144-5:       Users/Fakhri/AppData/Local
d/d 105196-144-1:       Users/Fakhri/AppData/Local/Application Data
d/d 36633-144-1:        Users/Fakhri/AppData/Local/Comms
d/d 36783-144-1:        Users/Fakhri/AppData/Local/Comms/Unistore
d/d 37628-144-1:        Users/Fakhri/AppData/Local/Comms/Unistore/data
r/r 107737-128-4:       Users/Fakhri/AppData/Local/Comms/Unistore/data/AggregateCache.uca
d/d 36637-144-6:        Users/Fakhri/AppData/Local/Comms/UnistoreDB
```

Command used:

1. fls -r -p -o 673792 FAKHRIWORKSTATION_20251211.E01 | grep "Users/Fakhri/"

Command explanation:

- -r → recursively searches through folders
- -p → displays the full file path
- -o 673792 → specifies the offset of the main partition

```
┌──(kali㉿kali)-[~/Downloads/DISKIMG_FAKRI251211]
└─$ fls -r -p -o 673792 FAKHRIWORKSTATION_20251211.E01 | grep "Users/Fakhri/Documents"
d/d 105084-144-6:       Users/Fakhri/Documents
r/r 110330-128-4:       Users/Fakhri/Documents/!!! DECRYPT_YOUR_FILES !!!.txt
r/r 103526-128-4:       Users/Fakhri/Documents/2024_Annual_Performance_Review.pdf.lock
r/r 119357-128-4:       Users/Fakhri/Documents/BigClient_Proposal_2025.docx.lock
r/r 105615-128-1:       Users/Fakhri/Documents/desktop.ini
r/r 120554-128-4:       Users/Fakhri/Documents/Favorite_Recipes.txt.lock
r/r 119355-128-4:       Users/Fakhri/Documents/Friday_Team_Meeting_Agenda.txt.lock
r/r 110412-128-4:       Users/Fakhri/Documents/Japan_Trip_2025_Planning.docx.lock
r/r 119358-128-4:       Users/Fakhri/Documents/Kids_Homework_Tracker.txt.lock
r/r 119356-128-4:       Users/Fakhri/Documents/Leadership_Meeting_Notes_Nov2024.txt.lock
d/d 105185-144-1:       Users/Fakhri/Documents/My Music
d/d 105186-144-1:       Users/Fakhri/Documents/My Pictures
d/d 105187-144-1:       Users/Fakhri/Documents/My Videos
r/r 119348-128-4:       Users/Fakhri/Documents/Q4_2024_Financial_Report.docx.lock
r/r 119347-128-4:       Users/Fakhri/Documents/TODO.txt.lock
r/r 119351-128-1:       Users/Fakhri/Documents/Vendor_Analysis_2024.xlsx
```

This search revealed several important files related to the incident. In particular, I found ransomware note files such as:

1. !!! DECRYPT_YOUR_FILES !!!.txt
2. BigClient_Proposal_2025.docx.lock

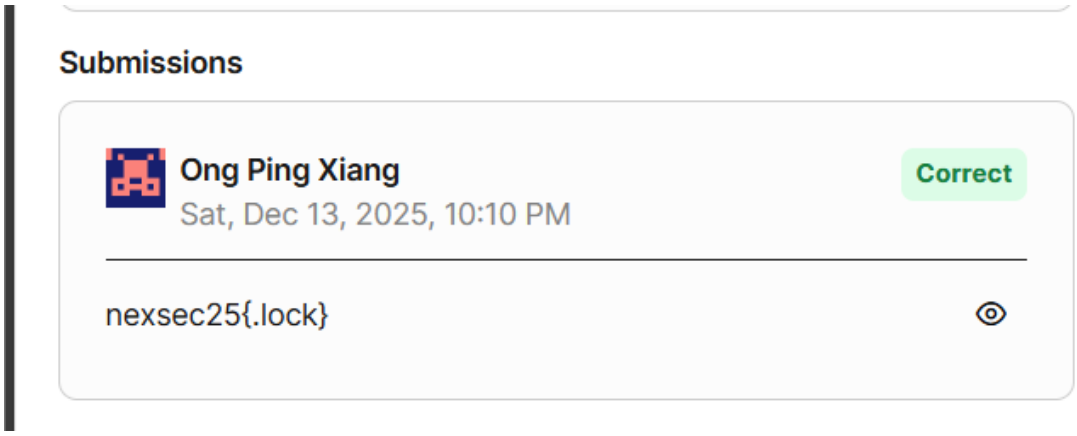
I also observed that the victim's personal files had been renamed with an additional extension.

One of the clearest examples was:

This shows that the ransomware appended the extension **.lock** to encrypted files.

Final Answer

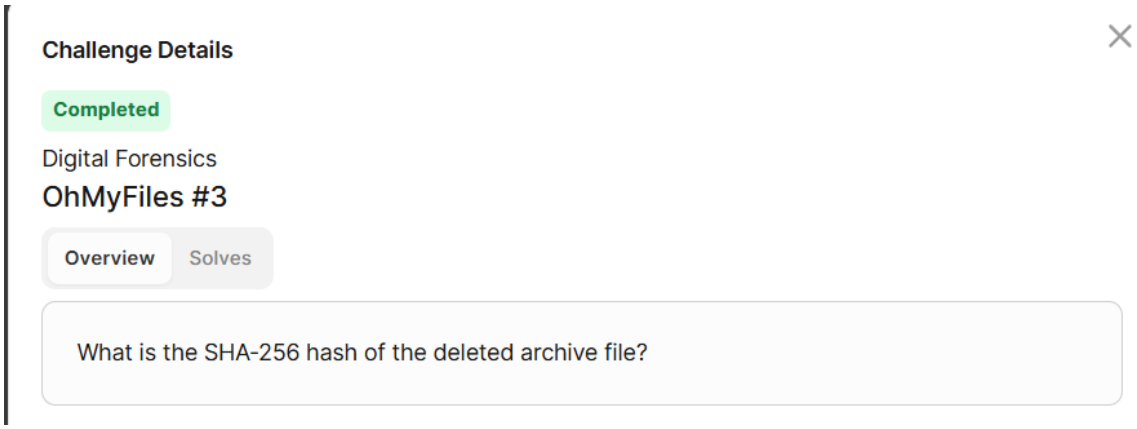
The file extension used for the encrypted files is: **.lock**



Flag: **nexsec25{.lock}**

OhMyFiles #3

Challenge Details



Main Task

Find the SHA-256 value of the deleted archive (.rar) file.

Initial Approach

Based on the incident report from the earlier challenge, the victim had downloaded a compressed archive and later deleted it after extraction. Since deleted files in Windows are often moved to the Recycle Bin, that was the first location I examined.

Step 1: Locate the Recycle Bin

I started by searching the disk image for the Recycle Bin directory.

```
┌──(kali㉿kali)-[~/Downloads/DISKIMG_FAKRI251211]
└─$ fls -r -o 673792 FAKHRIWORKSTATION_20251211.E01 | grep "Recycle"
++++ r/r 107716-128-1:  RecycleAndTradeInConfig.json
d/d 64-144-1:   $Recycle.Bin
++ r/r 38918-128-5:     Windows Recycle.wav
+++ r/r 38918-128-5:    Windows Recycle.wav
```

Command used:

1. fls -r -o 673792 FAKHRIWORKSTATION_20251211.E01 | grep "Recycle"

The command output confirmed the existence of the $Recycle.Bin directory with Inode 64.

1. d/d → Directory/Folder
2. r/r → Regular file

Next, I listed the contents of that directory.

```
┌──(kali㉿kali)-[~/Downloads/DISKIMG_FAKRI251211]
└─$ fls -o 673792 FAKHRIWORKSTATION_20251211.E01 64
d/d 104782-144-1:       S-1-5-18
d/d 106090-144-6:       S-1-5-21-1076420815-3808139571-653147571-1000
```

Command used:

1. fls -o 673792 FAKHRIWORKSTATION_20251211.E01 64

The output showed two subfolders:

- S-1-5-18

  This usually belongs to the system account and is often not useful for this kind of user activity.

- S-1-5-21-...-1000

  This is a user Security Identifier (SID). The ending 1000 typically refers to the first user account created on the system, which is likely the victim's account.

Since the deleted archive belonged to the user, I continued the investigation in the second folder.

Step 2: Identify the Deleted RAR File

Inside the user's recycle bin folder, I searched for archive files with the .rar extension.

```
┌──(kali㉿kali)-[~/Downloads/DISKIMG_FAKRI251211]
└─$ fls -o 673792 FAKHRIWORKSTATION_20251211.E01 106090
r/r 120585-128-1:       $I8CHBZQ.xlsx
r/r 120857-128-1:       $I96XXEK.rar
r/r 110861-128-1:       $IAGPFRN.lnk
r/r 120584-128-1:       $IOLGP11.xlsx
r/r 119833-128-1:       $ISF54LP.txt
r/r 119832-128-1:       $IY8T57Y.txt
r/r 119334-128-1:       $IZ4J5YN.txt
r/r 120560-128-1:       $IZT828V.xlsx
r/r 119349-128-4:       $R8CHBZQ.xlsx
r/r 110944-128-1:       $R96XXEK.rar
r/r 111049-128-3:       $RAGPFRN.lnk
r/r 119352-128-1:       $ROLGP11.xlsx
r/r 119359-128-4:       $RSF54LP.txt
r/r 119464-128-1:       $RY8T57Y.txt
r/r 34347-128-1:        $RZ4J5YN.txt
r/r 119346-128-1:       $RZT828V.xlsx
r/r 106091-128-1:       desktop.ini
```

Command used:

1. fls -o 673792 FAKHRIWORKSTATION_20251211.E01 106090

The output revealed two files:

- $I96XXEK.rar

  The $I file stores metadata about the deleted item, such as its original path and deletion details.

- $R96XXEK.rar

  The $R file contains the actual file data, which is the file needed for recovery.

The file $R96XXEK.rar had inode **110944**, so I used that inode to recover the deleted archive.

Step 3: Recover the Archive

I extracted the file data using icat and saved it as recovered_archive.rar.

```
┌──(kali㉿kali)-[~/Downloads/DISKIMG_FAKRI251211]
└─$ icat -o 673792 FAKHRIWORKSTATION_20251211.E01 110944 > recovered_archive.rar
```

```
┌──(kali㉿kali)-[~/Downloads/DISKIMG_FAKRI251211]
└─$ sha256sum recovered_archive.rar
cfaa2ce425e2f472618323dcbceb2e3fc013100919a8dbf545bf15b4c45dae8f  recovered_archive.rar
```

Command used:

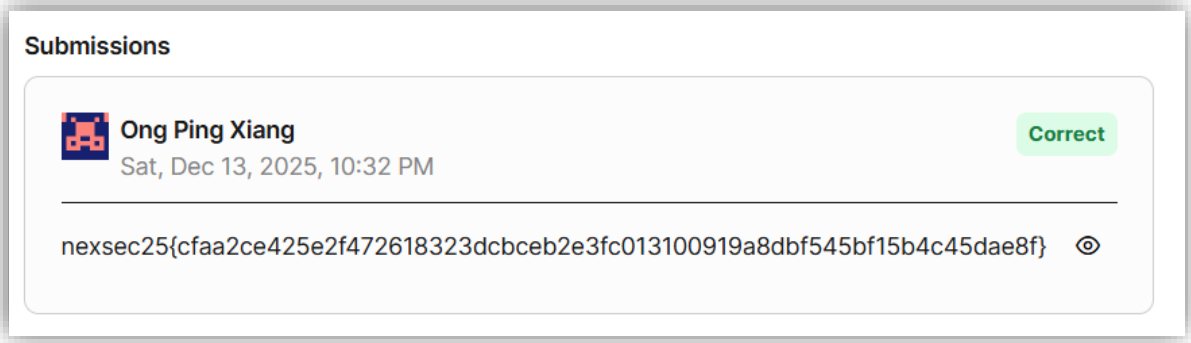
1. icat -o 673792 FAKHRIWORKSTATION_20251211.E01 110944 > recovered_archive.rar
2. sha256sum recovered_archive.rar

Once the archive was recovered, I calculated its SHA-256 hash value.

Final Answer

The SHA-256 value of the deleted archive is:

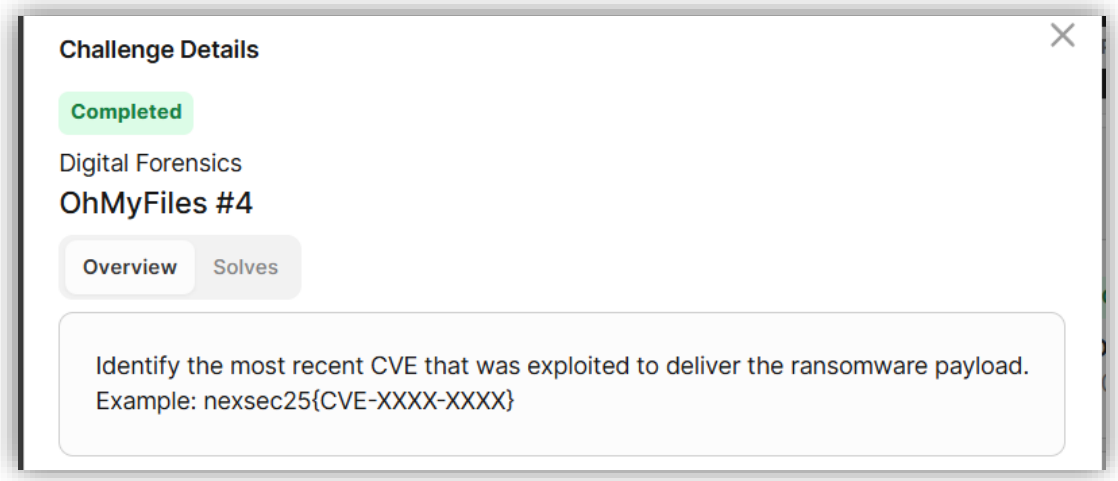
cfaa2ce425e2f472618323dcbceb2e3fc013100919a8dbf545bf15b4c45dae8f



Flag: **nexsec25{cfaa2ce425e2f472618323dcbceb2e3fc013100919a8dbf545bf15b4c45dae8f}**

OhMyFiles #4

Challenge Details



Main Task

Identify the most recent CVE that delivered the ransomware payload.

Initial Approach

In the previous challenge, I recovered a suspicious archive named recovered_archive.rar. To understand how the ransomware was delivered, I attempted to extract the archive in my Kali Linux virtual machine using 7z.

Although the extraction failed, the error messages turned out to be highly valuable. They revealed how the archive was crafted and pointed directly to the attack technique used.

Step 1: Inspect the Recovered Archive

```
┌──(kali㉿kali)-[~/Downloads/DISKIMG_FAKRI251211/analysis_folder]
└─$ 7z x recovered_archive.rar

7-Zip 25.01 (x64) : Copyright (c) 1999-2025 Igor Pavlov : 2025-08-03
 64-bit locale=en_US.UTF-8 Threads:32 OPEN_MAX:1024, ASM

Scanning the drive for archives:
1 file, 106408856 bytes (102 MiB)

Extracting archive: recovered_archive.rar
--
Path = recovered_archive.rar
Type = Rar5
Physical Size = 106408856
Characteristics = Locator QuickOpen:0
Encrypted = -
Solid = -
Blocks = 43
Method = v6:128K:m3
Multivolume = -
Volumes = 1


Would you like to replace the existing file:
  Path:     ./Resume_Template.docx
  Size:     2046 bytes (2 KiB)
  Modified: 2025-12-09 09:06:58
with the file from archive:
  Path:     Resume_Template.docx
  Size:     2046 bytes (2 KiB)
  Modified: 2025-12-09 09:06:58
? (Y)es / (N)o / (A)lways / (S)kip all / A(u)to rename all / (Q)uit? y

ERROR: Unsupported Method : Resume_Template.docx
ERROR: Unsupported Method : Resume_Template.docx:Windows\Temp00\Rar$86251.45885
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\AppData\Local\Temp\msedge.dll
ERROR: Unsupported Method : Resume_Template.docx:Windows\Temp02\Rar$13107.48576
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\AppData\Roaming\Microsoft\Win
dows\msedge.dll
ERROR: Unsupported Method : Resume_Template.docx:Windows\Temp04\Rar$12761.42348
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\ProgramData\msedge.dll
ERROR: Unsupported Method : Resume_Template.docx:Windows\Temp06\Rar$60812.48947
```

```
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\AppData\Local\Temp\msedge.dll
ERROR: Unsupported Method : Resume_Template.docx:Windows\Temp02\Rar$13107.48576
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\AppData\Roaming\Microsoft\Win
dows\msedge.dll
ERROR: Unsupported Method : Resume_Template.docx:Windows\Temp04\Rar$12761.42348
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\ProgramData\msedge.dll
ERROR: Unsupported Method : Resume_Template.docx:Windows\Temp06\Rar$60812.48947
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\..\ProgramData\msedge.dll
ERROR: Unsupported Method : Resume_Template.docx:Windows\Temp08\Rar$32994.40583
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\AppData\Roaming\Microsoft\
Windows\msedge.dll
ERROR: Unsupported Method : Resume_Template.docx:Windows\Temp10\Rar$42213.45750
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\ProgramData\msedge.dll
ERROR: Unsupported Method : Resume_Template.docx:..\..\AppData\Local\svchost.exe
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\AppData\Local\svchost.exe
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\AppData\Local\svchost.exe
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\AppData\Local\svchost.exe
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\..\AppData\Local\svchost.e
xe
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\..\..\AppData\Local\svchos
t.exe
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\..\..\..\AppData\Local\svc
host.exe
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\..\..\..\..\AppData\Local\
svchost.exe
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\..\..\..\..\..\AppData\Loc
al\svchost.exe
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\..\..\..\..\..\..\AppData\
Local\svchost.exe
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\..\..\..\..\..\..\..\..\..\AppDa
ta\Local\svchost.exe
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\..\..\..\..\..\..\..\..\..\..\Ap
pData\Local\svchost.exe
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\..\..\..\..\..\..\..\..\..\..\..
\..\AppData\Local\svchost.exe
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\..\..\..\..\..\..\..\..\..\..\..
\..\..\..\..\AppData\Local\svchost.exe
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\..\..\..\..\..\..\..\..\..\..\..
\..\..\..\..\..\..\AppData\Local\svchost.exe
ERROR: Unsupported Method : Resume_Template.docx:..\..\AppData\Roaming\Microsoft\Windows\S
tart Menu\Programs\Startup\startup.lnk
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\AppData\Roaming\Microsoft\Window
s\Start Menu\Programs\Startup\startup.lnk
```

```
   Date      Time    Attr         Size   Compressed  Name
------------------- ----- ------------ ------------  ------------------------
2025-12-09 09:06:58 ....A         2046         1957  Resume_Template.docx
------------------- ----- ------------ ------------  ------------------------
2025-12-09 09:06:58               2046         1957  1 files
2025-12-09 09:06:58          108847305    106399542  42 alternate streams
2025-12-09 09:06:58          108849351    106401499  43 streams
```

I tried to extract the recovered archive using the following command:

1. 7z x recovered_archive.rar

Instead of extracting normally, the command produced a large number of errors. However, a closer look at the output showed that these were not random failures. The archive entries contained suspicious file paths using ..\..\, which is a common indicator of a **path traversal** attack. It also showed that the .docx file was only about 2 KB, while the total archive size was about 108 MB, which was highly suspicious.

Step 2: Analyse the Malicious Archive Behaviour

From the error output, several important observations could be made:

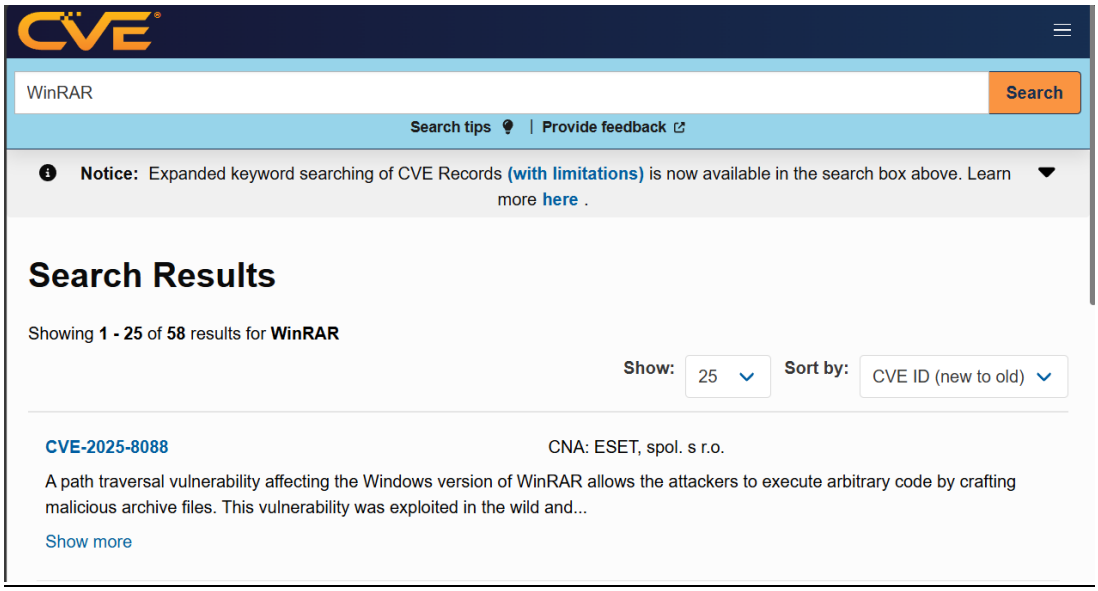
- Path Traversal Pattern
  The archive contained filenames with ..\..\, which means it was attempting to escape the intended extraction folder and write files into other locations on the system.

- Suspicious Payload Files
  The output showed attempts to drop files such as msedge.dll and startup.lnk. These names suggest the archive was designed to place malicious files into sensitive Windows directories.

- Why 7z Failed
  The 7z utility on Linux reported unsupported methods and failed to extract the archive properly. This suggests that the file was specially crafted for exploitation against Windows archive software, rather than for normal extraction in Linux.

These findings strongly indicated that the attacker was abusing a path traversal vulnerability in WinRAR to deliver the ransomware payload.

Step 3: Identify the Relevant CVE



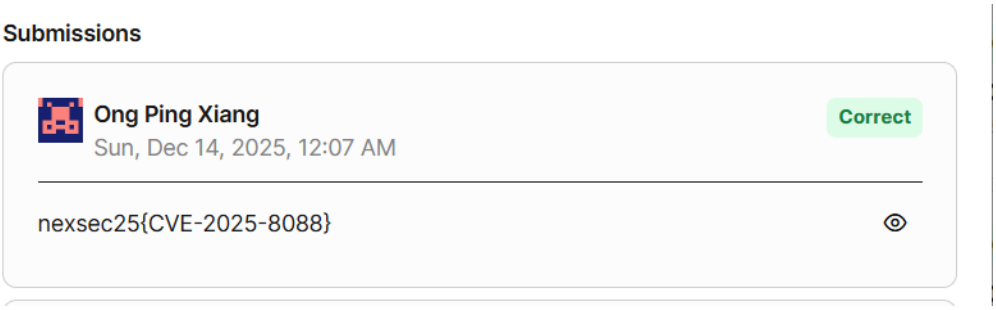
After identifying the likely attack method, I searched for recent WinRAR vulnerabilities related to path traversal and malicious archive extraction.

Based on the challenge context and the evidence observed in the archive, the most relevant match was:

- CVE ID: CVE-2025-8088
- Description: A path traversal vulnerability affecting the Windows version of WinRAR that allows attackers to execute arbitrary code through a crafted archive file.

This matched the behavior seen in the archive, where malicious files were designed to escape the extraction directory and land in locations such as the Startup folder and AppData.

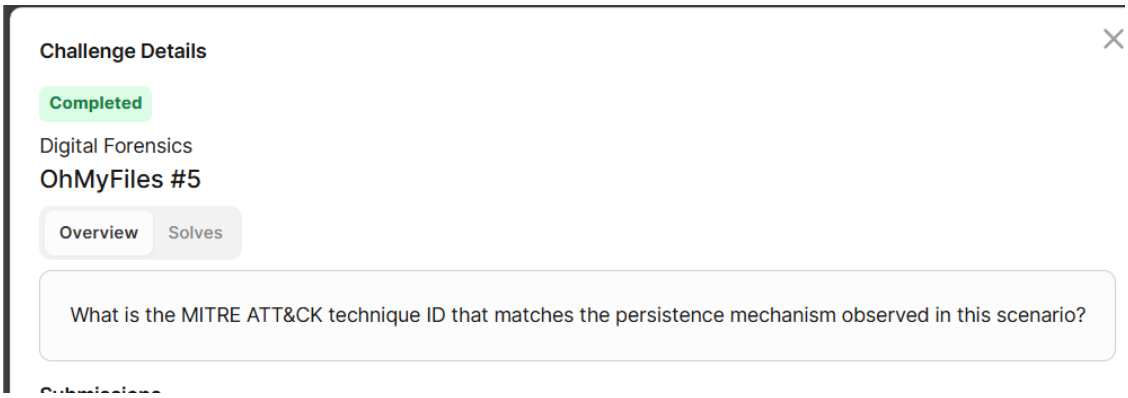
Final Answer



Flag: **nexsec25{CVE-2025-8088}**

OhMyFiles #5

Challenge Details



Main Task

Determine the MITRE ATT&CK technique ID that corresponds to the persistence mechanism used by the malicious archive.

Initial Approach

In the previous challenge, the recovered archive revealed suspicious path traversal entries when inspected with 7z. Among them, one of the most important findings was that the archive attempted to place a file named startup.lnk inside the user's **Startup** folder.

This behavior strongly suggested a persistence mechanism, since programs or shortcuts placed in the Windows Startup folder will automatically run when the user logs in.

Step 1: Identify the Persistence Behaviour

```
ERROR: Unsupported Method : Resume_Template.docx:..\..\AppData\Roaming\Microsoft\Windows\S
tart Menu\Programs\Startup\startup.lnk
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\AppData\Roaming\Microsoft\Window
s\Start Menu\Programs\Startup\startup.lnk
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\AppData\Roaming\Microsoft\Win
dows\Start Menu\Programs\Startup\startup.lnk
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\AppData\Roaming\Microsoft\
Windows\Start Menu\Programs\Startup\startup.lnk
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\..\AppData\Roaming\Microso
ft\Windows\Start Menu\Programs\Startup\startup.lnk
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\..\..\AppData\Roaming\Micr
osoft\Windows\Start Menu\Programs\Startup\startup.lnk
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\..\..\..\AppData\Roaming\M
icrosoft\Windows\Start Menu\Programs\Startup\startup.lnk
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\..\..\..\..\AppData\Roamin
g\Microsoft\Windows\Start Menu\Programs\Startup\startup.lnk
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\..\..\..\..\..\AppData\Roa
ming\Microsoft\Windows\Start Menu\Programs\Startup\startup.lnk
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\..\..\..\..\..\..\AppData\
Roaming\Microsoft\Windows\Start Menu\Programs\Startup\startup.lnk
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\..\..\..\..\..\..\..\..\AppDa
ta\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\startup.lnk
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\..\..\..\..\..\..\..\..\..\..\Ap
pData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\startup.lnk
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\..\..\..\..\..\..\..\..\..\..\..
\..\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\startup.lnk
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\..\..\..\..\..\..\..\..\..\..\..
\..\..\..\..\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\startup.lnk
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\..\..\..\..\..\..\..\..\..\..\..
\..\..\..\..\..\..\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\startup.l
nk
```

From the archive analysis, I observed that the malicious archive attempted to write the following file into the Startup directory: startup.lnk

In Windows, the **Startup** folder is a well-known location used to automatically launch programs after user logon. Because of this, attackers often abuse it to maintain persistence on an infected system.

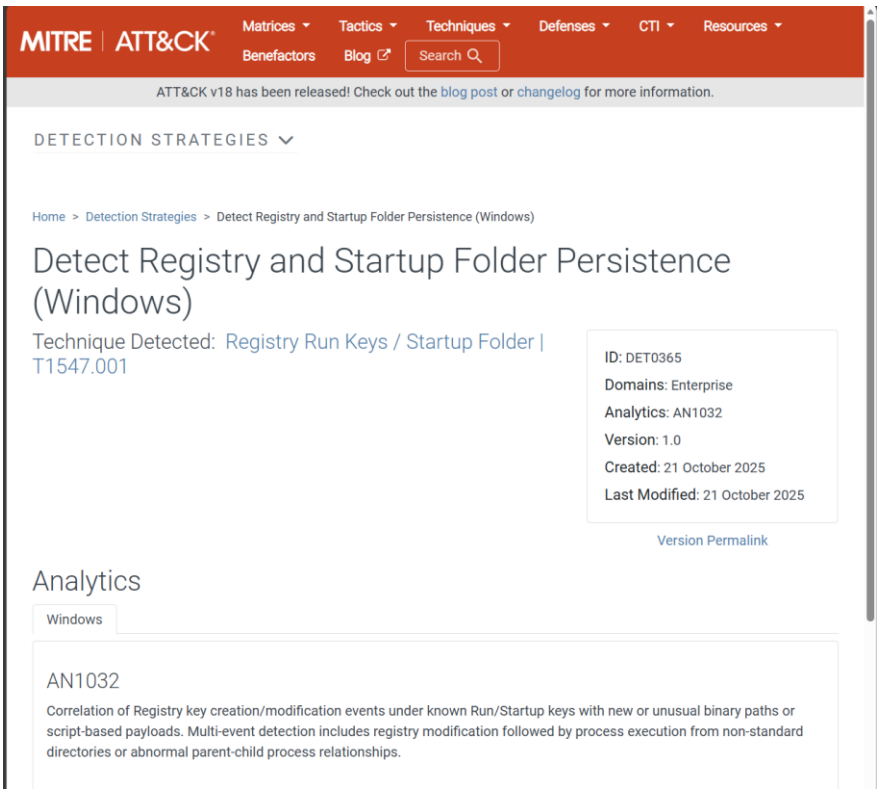
This means the ransomware was likely trying to ensure that its payload would continue to run even after the system was rebooted.

Step 2: Map the Behaviour to MITRE ATT&CK

To identify the correct technique ID, I referred to the **MITRE ATT&CK Framework**, which provides standardized IDs for common adversary behaviors.

I searched for techniques related to:

- Startup Folder
- Persistence
- Automatic execution on logon



The results pointed to the technique:

- Technique Name: Boot or Logon Autostart Execution
- Sub-technique: Registry Run Keys / Startup Folder
- Technique ID: T1547.001

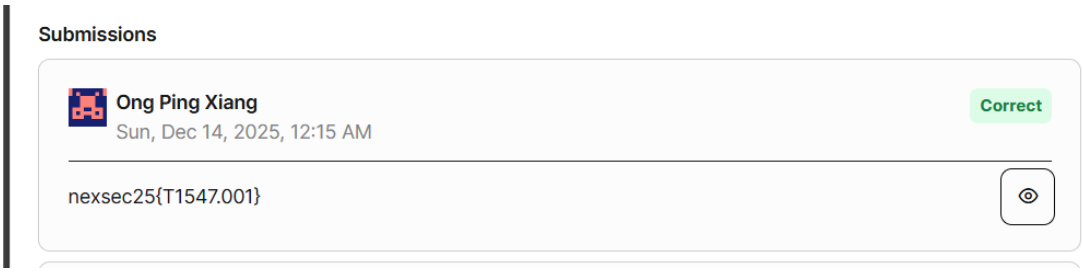
The MITRE ATT&CK description states that adversaries may achieve persistence by placing programs in a startup folder or by referencing them through registry run keys.

This matches the archive's attempt to drop startup.lnk into the Windows Startup folder.

Final Answer

The MITRE ATT&CK technique ID is:

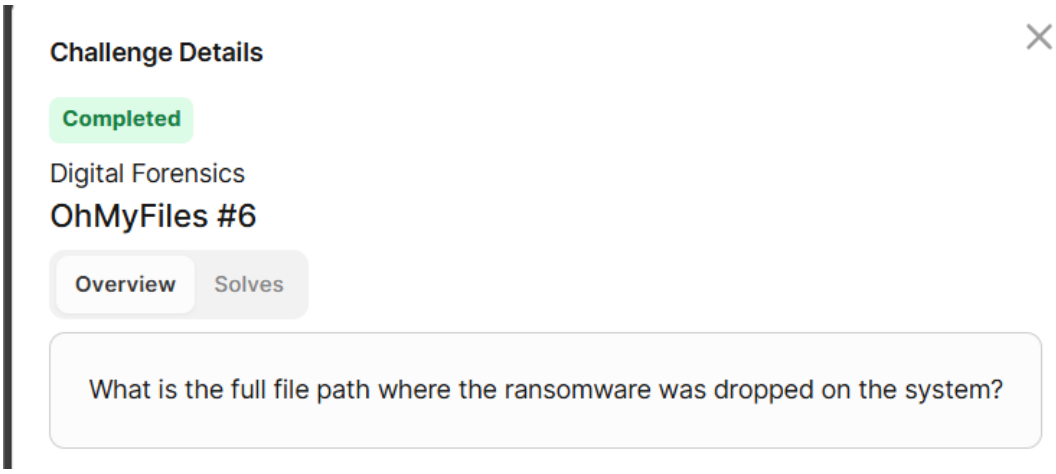
T1547.001



Flag: nexsec25{T1547.001}

OhMyFiles #6

Challenge Details



Main Task

Identify the full file path where the ransomware payload was dropped on the system.

Initial Approach

The answer to this challenge could be inferred from the archive analysis performed in the earlier challenge. When I attempted to inspect the malicious RAR file using 7z, the extraction failed and produced multiple error messages. Although the archive did not extract successfully, the output revealed important clues about where the malware intended to place its payload.

Step 1: Review the Archive Output

```
al\svchost.exe
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\..\..\..\..\..\..\..\AppData\
Local\svchost.exe
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\..\..\..\..\..\..\..\..\..\AppDa
ta\Local\svchost.exe
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\..\..\..\..\..\..\..\..\..\..\Ap
pData\Local\svchost.exe
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\..\..\..\..\..\..\..\..\..\..\..
\..\AppData\Local\svchost.exe
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\..\..\..\..\..\..\..\..\..\..\..
\..\..\..\..\..\AppData\Local\svchost.exe
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\..\..\..\..\..\..\..\..\..\..\..\..\..
\..\..\..\..\..\..\..\AppData\Local\svchost.exe
ERROR: Unsupported Method : Resume_Template.docx:..\..\AppData\Roaming\Microsoft\Windows\S
tart Menu\Programs\Startup\startup.lnk
ERROR: Unsupported Method : Resume_Template.docx:..\..\..\AppData\Roaming\Microsoft\Window
s\Start Menu\Programs\Startup\startup.lnk
```

During the inspection of recovered_archive.rar, I looked closely at the file paths shown in the 7z output, especially entries involving executable files.

From the error log, I observed that the archive attempted to place a file named:

svchost.exe

into the following Windows user directory:

AppData\Local

This strongly suggested that the ransomware payload was intended to be dropped into the victim's local application data folder.

Step 2: Reconstruct the Full Path

To determine the complete file path, I combined the evidence gathered from earlier analysis:

- Drive Letter
  The system uses the standard Windows drive letter C:.
- Username
  Previous investigation of the disk image confirmed that the victim's user profile was under Users\Fakhri.
- Drop Location
  The archive output indicated that the executable was targeting AppData\Local\svchost.exe.

Combining these parts gives the full path:

C:\Users\Fakhri\AppData\Local\svchost.exe

Step 3: Forensic Observation

It is worth noting that the filename **svchost.exe** was likely chosen deliberately to appear legitimate.

In Windows, svchost.exe is a real system process. However, the legitimate file normally runs from:

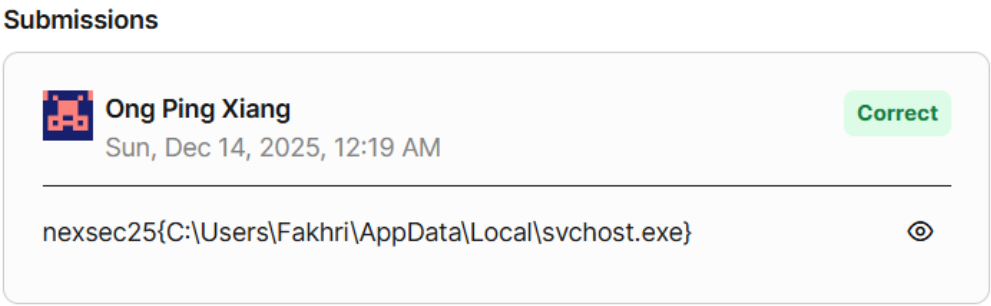
C:\Windows\System32\svchost.exe

In this case, the payload was placed inside the user's AppData\Local folder instead. This is suspicious and indicates an attempt to **masquerade** as a normal Windows process in order to avoid attention.

Final Answer

The full file path where the ransomware payload was dropped is:

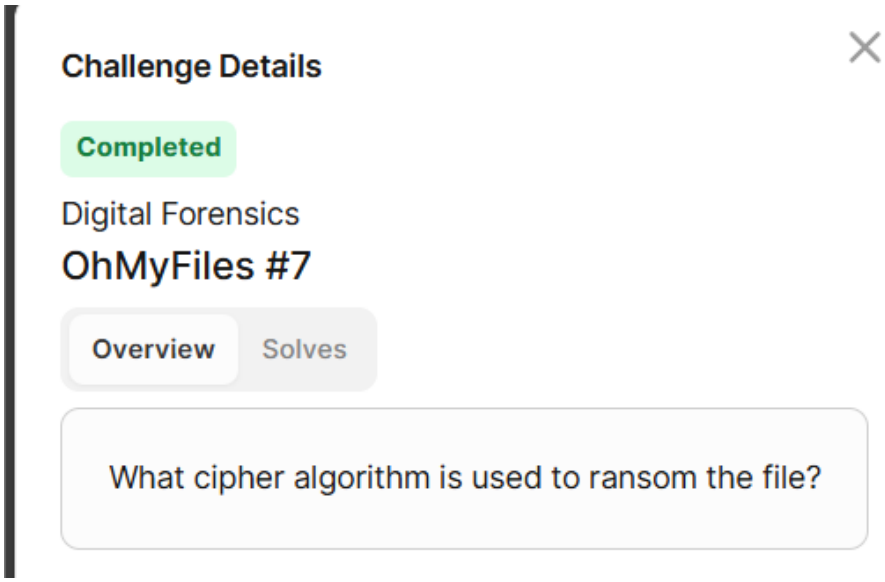
C:\Users\Fakhri\AppData\Local\svchost.exe



Flag: **nexsec25{C:\Users\Fakhri\AppData\Local\svchost.exe}**

OhMyFiles #7

Challenge Details



Main Task

Determine the cipher algorithm used by the ransomware to encrypt the files.

Initial Approach

This challenge became clearer after noticing a hint from the next challenge, which mentioned the Windows registry. That suggested the ransomware might have stored its configuration or encryption details inside the user's registry hive.

To investigate this, I searched for the victim's NTUSER.DAT file in the forensic disk image. Since NTUSER.DAT stores per-user registry settings, it was a good place to look for ransomware-related entries.

Step 1: Locate the User Registry Hive

I began by searching the disk image for NTUSER.DAT.

```
┌──(kali㉿kali)-[~/Downloads/DISKIMG_FAKRI251211]
└─$ fls -r -o 673792 FAKHRIWORKSTATION_20251211.E01 | grep "NTUSER.DAT"
++ r/r 102552-128-1:    NTUSER.DAT{53b39e88-18c4-11ea-a811-000d3aa4692b}.TM.blf
++ r/r 102532-128-3:    NTUSER.DAT
++ r/r 102531-128-4:    NTUSER.DAT.LOG1
++ r/r 36273-128-4:     NTUSER.DAT.LOG2
++ r/r 102553-128-1:    NTUSER.DAT{53b39e88-18c4-11ea-a811-000d3aa4692b}.TMContainer000000
00000000000001.regtrans-ms
++ r/r 102554-128-1:    NTUSER.DAT{53b39e88-18c4-11ea-a811-000d3aa4692b}.TMContainer000000
00000000000002.regtrans-ms
++ r/r 105175-128-1:    NTUSER.DAT{53b39e88-18c4-11ea-a811-000d3aa4692b}.TM.blf
++ r/r 105076-128-4:    NTUSER.DAT
++ r/r 105176-128-1:    NTUSER.DAT{53b39e88-18c4-11ea-a811-000d3aa4692b}.TMContainer000000
00000000000001.regtrans-ms
++ r/r 105177-128-1:    NTUSER.DAT{53b39e88-18c4-11ea-a811-000d3aa4692b}.TMContainer000000
00000000000002.regtrans-ms
+++ r/r 102804-128-3:   NTUSER.DAT
+++ r/r 102805-128-4:   NTUSER.DAT.LOG1
+++ r/r 102806-128-4:   NTUSER.DAT.LOG2
+++ r/r 102810-128-1:   NTUSER.DAT{53b39e88-18c4-11ea-a811-000d3aa4692b}.TM.blf
+++ r/r 102811-128-1:   NTUSER.DAT{53b39e88-18c4-11ea-a811-000d3aa4692b}.TMContainer000000
00000000000001.regtrans-ms
```

Command used:

1. fls -r -o 673792 FAKHRIWORKSTATION_20251211.E01 | grep "NTUSER.DAT"

The output showed multiple NTUSER.DAT files and related log files. Because the disk image contained more than one possible hive, I had to identify the correct one for the active user profile.

From the results, several candidate inodes were observed, including 102532, 105076, and 102804. After testing, the hive associated with inode 105076 contained the ransomware configuration needed for this challenge.

Step 2: Extract the Registry Hive

Once the correct hive was identified, I extracted it from the disk image using icat.

```
┌──(kali㉿kali)-[~/Downloads/DISKIMG_FAKRI251211]
└─$ icat -o 673792 FAKHRIWORKSTATION_20251211.E01 105076 > NTUSER.DAT
```

Command used:

1. icat -o 673792 FAKHRIWORKSTATION_20251211.E01 105076 > NTUSER.DAT

This recovered the user registry hive so it could be examined locally.

Step 3: Search for Ransomware Configuration

```
═══════════════════════════════════════════════════════════════════════════
                    SHADOWCRYPT RANSOMWARE
═══════════════════════════════════════════════════════════════════════════

YOUR FILES HAVE BEEN ENCRYPTED!

All your important documents are now locked with .lock extension.
Without our decryption key, recovery is IMPOSSIBLE.

───────────────────────────────────────────────────────────────────────────
WHAT HAPPENED?
───────────────────────────────────────────────────────────────────────────
Location: Documents folder
File Extension: .lock

───────────────────────────────────────────────────────────────────────────
WARNING - TIME SENSITIVE!
───────────────────────────────────────────────────────────────────────────
You have 72 HOURS to pay before your decryption key is deleted forever.

Price increases every 24 hours:
  0-24 hours:  0.5 BTC
  24-48 hours: 0.6 BTC
  48-72 hours: 0.7 BTC
  After 72h:   FILES LOST PERMANENTLY

═══════════════════════════════════════════════════════════════════════════
                SHADOWCRYPT TEAM - Since 2019
           "Your files. Our encryption. Your choice."
═══════════════════════════════════════════════════════════════════════════

NOTE: This is a CTF challenge for educational purposes only.
═══════════════════════════════════════════════════════════════════════════
```

Since the ransom notes previously referenced the name **ShadowCrypt**, I used reglookup to search the extracted hive for entries related to that name.

```
┌──(kali㉿kali)-[~/Downloads/DISKIMG_FAKRI251211]
└─$ reglookup NTUSER.DAT | grep -i "ShadowCrypt"
WARN: While quoting value for '/SOFTWARE/Microsoft/Windows/CurrentVersion/ActivityDataModel
/ReaderRevisionInfo/183B5956-BC23-BAD2-F6E6-3B703DAD42A4', warning returned: Data could not
 be interpreted, quoting raw buffer.
/SOFTWARE/ShadowCrypt,KEY,,2025-12-11 06:08:17
/SOFTWARE/ShadowCrypt/Info,KEY,,2025-12-11 06:08:17
/SOFTWARE/ShadowCrypt/Info/Encrypted,SZ,9,
/SOFTWARE/ShadowCrypt/Info/Method,SZ,XOR_MD5_YEAR,
/SOFTWARE/ShadowCrypt/Info/Timestamp,SZ,2025-12-11T14:08:17.362668,
/SOFTWARE/ShadowCrypt/Keys,KEY,,2025-12-11 06:08:17
/SOFTWARE/ShadowCrypt/Keys/Favorite_Recipes.txt.lock,SZ,ab4ee9472f07a047c4eec308383479ee202
5,
/SOFTWARE/ShadowCrypt/Keys/Friday_Team_Meeting_Agenda.txt.lock,SZ,00e12adebe98b01163c51f58b
00c416b2025,
/SOFTWARE/ShadowCrypt/Keys/Kids_Homework_Tracker.txt.lock,SZ,24409ce509cd69b904651fadd18080
412025,
/SOFTWARE/ShadowCrypt/Keys/Leadership_Meeting_Notes_Nov2024.txt.lock,SZ,55ddc8e80569fc9120c
cf05b5fdb7c232025,
/SOFTWARE/ShadowCrypt/Keys/TODO.txt.lock,SZ,d695ba7862c9e6718ca09b676b12ebad2025,
/SOFTWARE/ShadowCrypt/Keys/BigClient_Proposal_2025.docx.lock,SZ,d39316995b8fdc7ccbd7662b44b
b374c2025,
/SOFTWARE/ShadowCrypt/Keys/Japan_Trip_2025_Planning.docx.lock,SZ,446613e99cef39a7b7ec7c76cb
32c9952025,
/SOFTWARE/ShadowCrypt/Keys/Q4_2024_Financial_Report.docx.lock,SZ,a4ab0c98a145d242fa51f959be
aeaa172025,
/SOFTWARE/ShadowCrypt/Keys/2024_Annual_Performance_Review.pdf.lock,SZ,4fcfcc891575cfc96d34d
0bcb94d5d462025,
WARN: While quoting value for '/System/GameConfigStore/Children/dffdaf75-db59-48b9-872f-91b
aa438647d/ExeParentDirectory', warning returned: Data could not be interpreted, quoting raw
 buffer.
```

Command used:

1. reglookup NTUSER.DAT | grep -i "ShadowCrypt"

The output revealed several registry entries created by the ransomware. One of the most important lines was:

SOFTWARE\ShadowCrypt\Info\Method, SZ, XOR_MD5_YEAR

This entry exposed the ransomware's encryption method. The value suggests that the ransomware used **XOR** as the main cipher, while **MD5** may have been involved in generating or deriving the key.

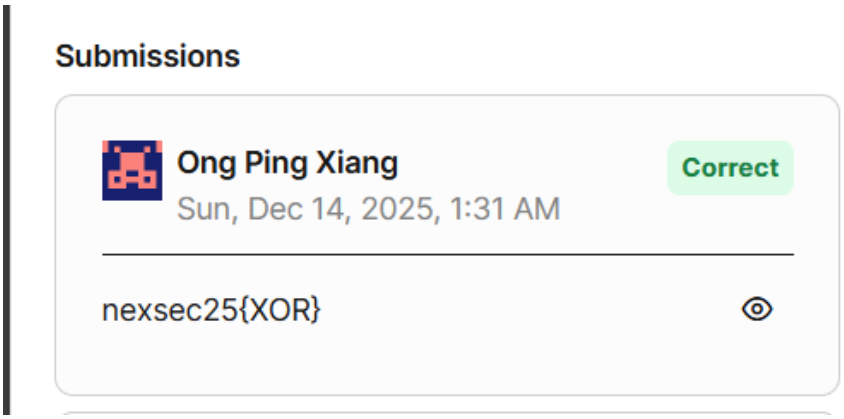
Since the challenge specifically asked for the cipher algorithm, the relevant answer is:

XOR

Final Answer

The cipher algorithm used by the ransomware is:

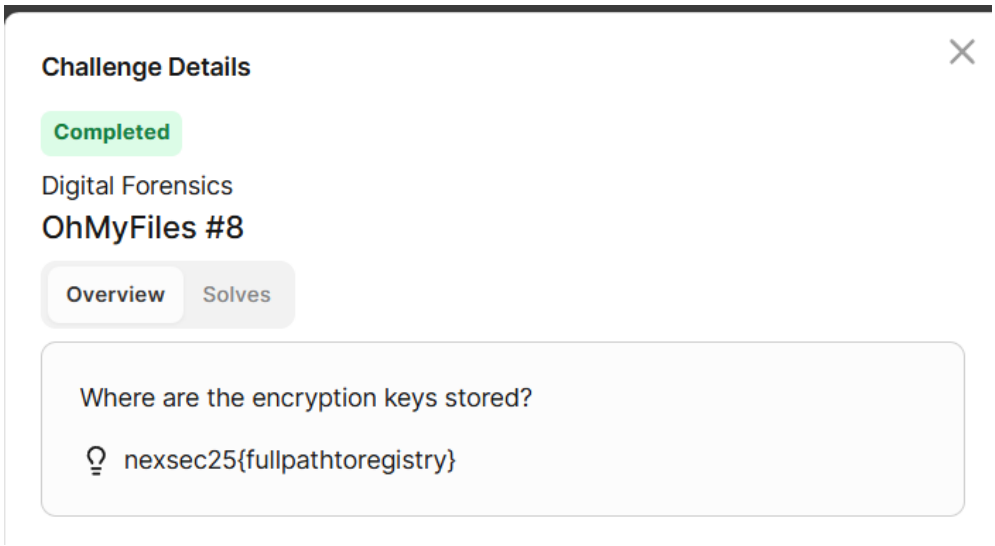
XOR



Flag: **nexsec25{XOR}**

OhMyFiles #8

Challenge Details



Main Task

Determine the full registry key path where the ShadowCrypt ransomware stores its encryption keys and configuration.

Initial Approach

In the previous challenge, I examined the victim's NTUSER.DAT registry hive and searched for entries related to **ShadowCrypt**. The output revealed multiple ransomware-related registry values, including entries associated with encrypted files.

To answer this challenge, I focused on identifying the exact registry location where those values were stored.

Step 1: Review the Registry Output

```
┌──(kali㉿kali)-[~/Downloads/DISKIMG_FAKRI251211]
└─$ reglookup NTUSER.DAT | grep -i "ShadowCrypt"
WARN: While quoting value for '/SOFTWARE/Microsoft/Windows/CurrentVersion/ActivityDataModel
/ReaderRevisionInfo/183B5956-BC23-BAD2-F6E6-3B703DAD42A4', warning returned: Data could not
 be interpreted, quoting raw buffer.
/SOFTWARE/ShadowCrypt,KEY,,2025-12-11 06:08:17
/SOFTWARE/ShadowCrypt/Info,KEY,,2025-12-11 06:08:17
/SOFTWARE/ShadowCrypt/Info/Encrypted,SZ,9,
/SOFTWARE/ShadowCrypt/Info/Method,SZ,XOR_MD5_YEAR,
/SOFTWARE/ShadowCrypt/Info/Timestamp,SZ,2025-12-11T14:08:17.362668,
/SOFTWARE/ShadowCrypt/Keys,KEY,,2025-12-11 06:08:17
/SOFTWARE/ShadowCrypt/Keys/Favorite_Recipes.txt.lock,SZ,ab4ee9472f07a047c4eec308383479ee202
5,
/SOFTWARE/ShadowCrypt/Keys/Friday_Team_Meeting_Agenda.txt.lock,SZ,00e12adebe98b01163c51f58b
00c416b2025,
/SOFTWARE/ShadowCrypt/Keys/Kids_Homework_Tracker.txt.lock,SZ,24409ce509cd69b904651fadd18080
412025,
/SOFTWARE/ShadowCrypt/Keys/Leadership_Meeting_Notes_Nov2024.txt.lock,SZ,55ddc8e80569fc9120c
cf05b5fdb7c232025,
/SOFTWARE/ShadowCrypt/Keys/TODO.txt.lock,SZ,d695ba7862c9e6718ca09b676b12ebad2025,
/SOFTWARE/ShadowCrypt/Keys/BigClient_Proposal_2025.docx.lock,SZ,d39316995b8fdc7ccbd7662b44b
b374c2025,
/SOFTWARE/ShadowCrypt/Keys/Japan_Trip_2025_Planning.docx.lock,SZ,446613e99cef39a7b7ec7c76cb
32c9952025,
/SOFTWARE/ShadowCrypt/Keys/Q4_2024_Financial_Report.docx.lock,SZ,a4ab0c98a145d242fa51f959be
aeaa172025,
/SOFTWARE/ShadowCrypt/Keys/2024_Annual_Performance_Review.pdf.lock,SZ,4fcfcc891575cfc96d34d
0bcb94d5d462025,
WARN: While quoting value for '/System/GameConfigStore/Children/dffdaf75-db59-48b9-872f-91b
aa438647d/ExeParentDirectory', warning returned: Data could not be interpreted, quoting raw
 buffer.
```

Using the reglookup output from the previous challenge, I observed that several encrypted files were listed under the following registry path:

SOFTWARE\ShadowCrypt\Keys

Examples included entries related to encrypted documents such as:

- Favorite_Recipes.txt.lock
- Friday_Team_Meeting_Agenda.txt.lock

This strongly suggested that the **Keys** subkey was being used by the ransomware to store encryption-related data or metadata for individual encrypted files.

Step 2: Identify the Full Registry Path

The path shown in the output came from the extracted NTUSER.DAT hive, which corresponds to the current user's registry settings in Windows.

Forensically, this means the root hive should be written as:

HKEY_CURRENT_USER

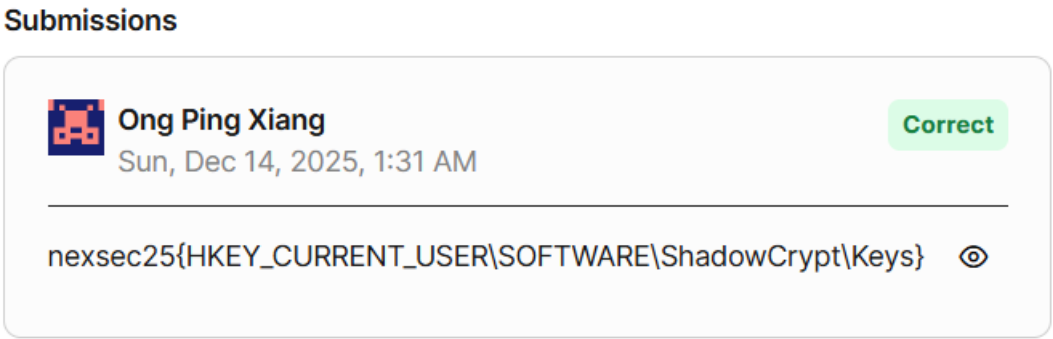
When combined with the subkey identified in the output, the full registry path becomes:

HKEY_CURRENT_USER\SOFTWARE\ShadowCrypt\Keys

Final Answer

The full registry key path is:

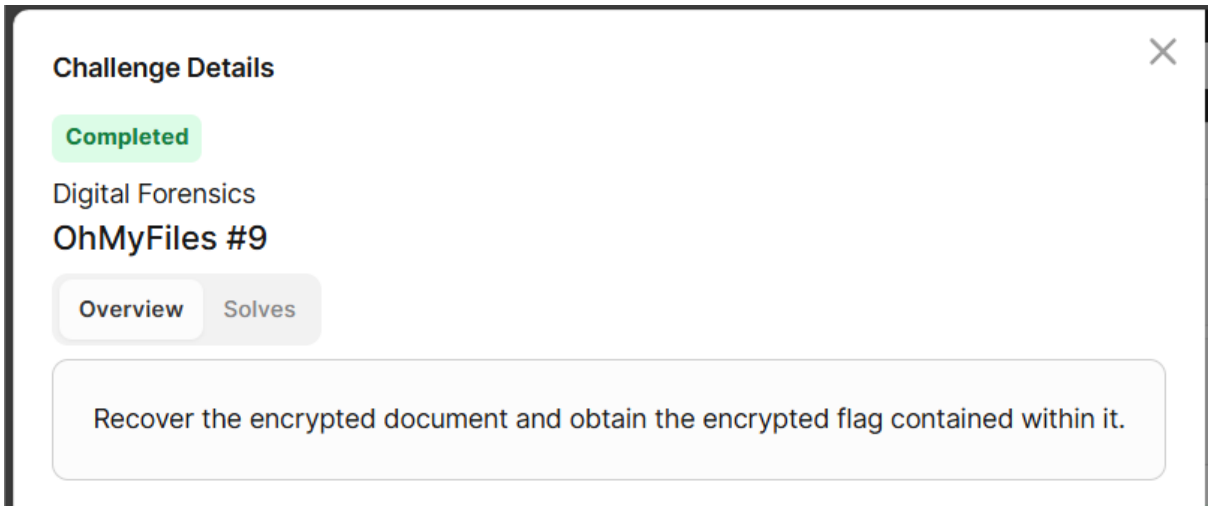
HKEY_CURRENT_USER\SOFTWARE\ShadowCrypt\Keys



Flag: nexsec25{HKEY_CURRENT_USER\SOFTWARE\ShadowCrypt\Keys}

OhMyFiles #9

Challenge Details



Main Task

Recover the encrypted document and obtain the flag hidden inside it.

Initial Approach

From the previous challenge, I had already identified the registry location used by the ShadowCrypt ransomware to store file-related encryption data. That registry information included the entry for the important document mentioned earlier in the incident report:

BigClient_Proposal_2025.docx.lock

This suggested that both the encrypted file and its associated key material could be recovered from the forensic image and used to restore the original content.

Step 1: Identify the Encrypted Document

I first located the encrypted version of the important file inside the disk image. Since the ransomware had renamed victim files with the .lock extension, I searched for:

BigClient_Proposal_2025.docx.lock

Once the file was located, it was extracted from the forensic image for further analysis.

Step 2: Use the Registry Data for Decryption

```
/SOFTWARE/ShadowCrypt/Keys/BigClient_Proposal_2025.docx.lock,SZ,d39316995b8fdc7ccbd7662b44b
b374c2025,
```

From the earlier registry analysis of NTUSER.DAT, I had already recovered the encryption key associated with the locked file. This value was then used as part of the decryption process.

At this stage, the objective was to restore the contents of the encrypted document rather than just identify metadata about it.

Step 3: Decrypt the File and Inspect the Contents

```
┌──(kali㉿kali)-[~/Downloads/DISKIMG_FAKRI251211]
└─$ fls -o 673792 -r -p FAKHRIWORKSTATION_20251211.E01 | grep "BigClient"
r/r 119822-128-4:       Users/Fakhri/AppData/Roaming/Microsoft/Windows/Recent/BigClient_Proposal_
2025.docx.lnk
r/r 119357-128-4:       Users/Fakhri/Documents/BigClient_Proposal_2025.docx.lock
^C

┌──(kali㉿kali)-[~/Downloads/DISKIMG_FAKRI251211]
└─$ icat -o 673792 FAKHRIWORKSTATION_20251211.E01 119357 > BigClient_Proposal_2025.docx.lock
```

After extracting the locked document, I used a decryption script to recover its contents.

```
kali@kali: ~/Downloads/DISKIMG_FAKRI251211
Session  Actions  Edit  View  Help
  GNU nano 8.6                                  decrypt.py
import itertools

# 1. 设置输入输出文件名
input_file = "BigClient_Proposal_2025.docx.lock"
output_file = "BigClient_Proposal_2025.docx"

# 2. 设置你在注册表里找到的那个密钥！
# 注意：这只是字符串，不需要 hex 解码，直接当字符串用
key_string = "d39316995b8fdc7ccbd7662b44bb374c2025"

# 将密钥字符串转换为字节
key = key_string.encode('utf-8')

def xor_decrypt(data, key):
    # 使用 itertools.cycle 循环使用密钥
    return bytearray([b ^ k for b, k in zip(data, itertools.cycle(key))])

try:
    # 读取加密文件
    with open(input_file, "rb") as f:
        encrypted_data = f.read()

    # 解密
    decrypted_data = xor_decrypt(encrypted_data, key)

    # 保存解密后的文件
    with open(output_file, "wb") as f:
        f.write(decrypted_data)

    print(f"✅ Success! File decrypted to: {output_file}")
    print("Now try to open it or grep for the flag!")

except Exception as e:
    print(f"❌ Error: {e}")
```

Once the file was decrypted, I examined the output and searched for the challenge flag embedded inside the recovered document data.

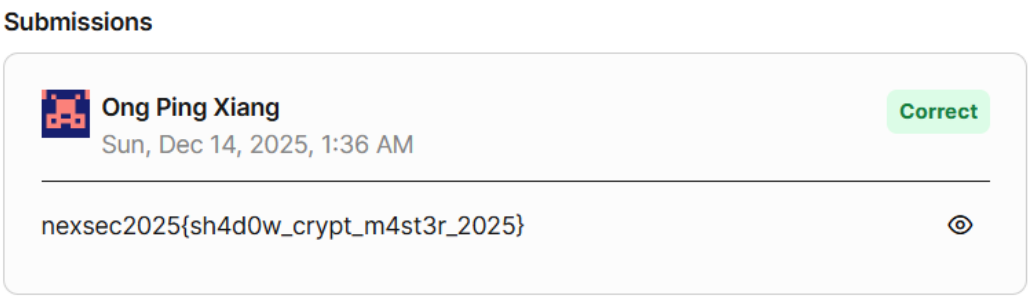
(I used an AI-generated script to assist with the decryption step.)

```
┌──(kali㉿kali)-[~/Downloads/DISKIMG_FAKRI251211]
└─$ python3 decrypt.py
✅ Success! File decrypted to: BigClient_Proposal_2025.docx
Now try to open it or grep for the flag!

┌──(kali㉿kali)-[~/Downloads/DISKIMG_FAKRI251211]
└─$
grep -a "nexsec" BigClient_Proposal_2025.docx
nexsec2025{sh4d0w_crypt_m4st3r_2025}
```

The decrypted content revealed the following flag:

nexsec2025{sh4d0w_crypt_m4st3r_2025}

Final Answer

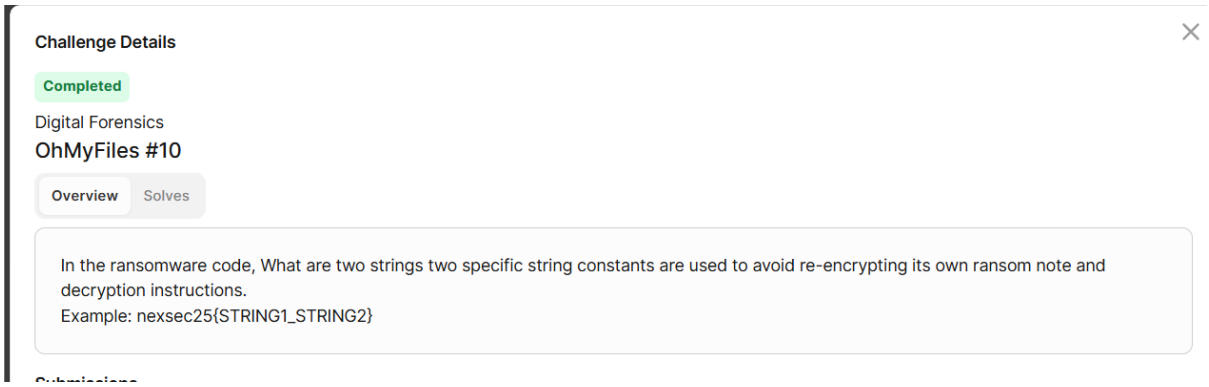


The recovered flag from the encrypted document is:

nexsec2025{sh4d0w_crypt_m4st3r_2025}

OhMyFiles #10

Challenge Details



Main Task

Identify the two string constants used by the ransomware to avoid re-encrypting its own ransom note and decryption instructions.

Initial Approach

To solve this challenge, I needed to inspect the ransomware code itself. From the earlier analysis, I already knew that the malicious archive dropped a fake Resume_Template.docx file and a hidden payload related to svchost.exe.

Since the challenge specifically asked for two string constants inside the ransomware code, the next step was to recover the embedded executable and inspect how it was built.

Step 1: Extract the Hidden Payload from the Document

I first copied the recovered archive into my Windows virtual machine and extracted the malicious Resume_Template.docx file. From earlier challenges, this file had already appeared suspicious because it was used to disguise the ransomware payload.

The payload was stored inside the document as an alternate data stream, so I used PowerShell to extract the hidden bytes and write them into a separate file.

```
FLARE-VM 04/24/2026 01:34:11
PS C:\Users\kali\Desktop\ohmyfiles_out > $bytes = Get-Content -Path .\Resume_Template.d
ocx -Stream $target.Stream -Encoding Byte -ReadCount 0
>> [System.IO.File]::WriteAllBytes("C:\Users\kali\Desktop\ohmyfiles_out\svchost_dump.bi
n", $bytes)
FLARE-VM 04/24/2026 01:34:23
PS C:\Users\kali\Desktop\ohmyfiles_out > ls


    Directory: C:\Users\kali\Desktop\ohmyfiles_out


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         22/4/2026   2:49 PM      106408856 recovered_archive.rar
------         9/12/2025  10:06 PM           2046 Resume_Template.docx
-a----         24/4/2026   1:34 AM        7254759 svchost_dump.bin


FLARE-VM 04/24/2026 01:34:25
PS C:\Users\kali\Desktop\ohmyfiles_out >
```

Command used:

1. $bytes = Get-Content -Path .\Resume_Template.docx -Stream $target.Stream -Encoding Byte -ReadCount 0 [System.IO.File]::WriteAllBytes("C:\Users\kali\Desktop\ohmyfiles_out\svchost_dump.bin", $bytes)

This recovered the embedded binary data as:

svchost_dump.bin I then renamed it to svchost_dump.exe so it could be analysed more easily as a Windows executable.

Step 2: Inspect the Executable in Kali Linux

After recovering the executable, I moved it back into my Kali Linux virtual machine for static analysis.

I first checked the strings inside the file and noticed several references related to **PyInstaller**.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ strings -a -el svchost_dump.exe | grep -Ei 'shadow|lock|xor|md5|decrypt|note|startup|msedg
e|svchost|pyi|pyinstaller|meipass'
[PYI-%d:%ls]
[PYI-%d:ERROR]
PyInstallerOnefileHiddenWindow
PyInstaller Onefile Hidden Window
Security descriptor string length exceeds PYI_PATH_MAX!
```

Command used:

1. strings -a -el svchost_dump.exe | grep -Ei 'shadow|lock|xor|md5|decrypt|note|startup|msedge|svchost|pyi|pyinstaller|meipass'

The output contained strings such as:

- PyInstaller
- pyi-windows-manifest-filename
- _MEIPASS

These indicators strongly suggested that the attacker had used **PyInstaller** to package a Python-based ransomware script into a standalone Windows .exe file.

Step 3: Why PyInstaller Matters

This is an important point for the write-up.

PyInstaller is a tool that converts Python scripts into standalone executables. Instead of distributing a plain .py script, an attacker can package the code together with the Python interpreter and its dependencies into a single .exe file.

From a forensic perspective, this explains why:

- the malware looked like a normal Windows executable
- the actual Python source code was not directly visible at first
- the file needed to be unpacked before its logic could be inspected

In other words, the ransomware was not originally written as a native Windows binary. It was most likely a Python program that had been bundled into an executable using PyInstaller.

Step 4: Extract the PyInstaller Archive

Since the executable appeared to be built with PyInstaller, I used **pyinstxtractor** to unpack it.

```
You can now use a python decompiler on the pyc files within the extracted directory

┌──(kali㉿kali)-[~/Desktop]
└─$ python3 pyinstxtractor.py svchost_dump.exe
[+] Processing svchost_dump.exe
[+] Pyinstaller version: 2.1+
[+] Python version: 3.13
[+] Length of package: 6915303 bytes
[+] Found 21 files in CArchive
[+] Beginning extraction...please standby
[+] Possible entry point: pyiboot01_bootstrap.pyc
[+] Possible entry point: pyi_rth_inspect.pyc
[+] Possible entry point: svchost.pyc
[+] Found 112 files in PYZ archive
[+] Successfully extracted pyinstaller archive: svchost_dump.exe

You can now use a python decompiler on the pyc files within the extracted directory
```

Command used:

1. wget -O pyinstxtractor.py https://raw.githubusercontent.com/extremecoders-re/pyinstxtractor/master/pyinstxtractor.py
2. python3 pyinstxtractor.py svchost_dump.exe

The extraction succeeded and produced a folder named:

svchost_dump.exe_extracted

```
┌──(kali㉿kali)-[~/Desktop]
└─$ cd svchost_dump.exe_extracted

┌──(kali㉿kali)-[~/Desktop/svchost_dump.exe_extracted]
└─$ ls
base_library.zip  pyiboot01_bootstrap.pyc  python313.dll      svchost.pyc
_bz2.pyd          pyimod01_archive.pyc     PYZ.pyz            unicodedata.pyd
_decimal.pyd      pyimod02_importers.pyc   PYZ.pyz_extracted  VCRUNTIME140.dll
_hashlib.pyd      pyimod03_ctypes.pyc      select.pyd
libcrypto-3.dll   pyimod04_pywin32.pyc     _socket.pyd
_lzma.pyd         pyi_rth_inspect.pyc      struct.pyc
```

Inside that folder, several files were recovered, including:

- svchost.pyc
- PYZ.pyz
- other PyInstaller support files and Python runtime components

The presence of svchost.pyc indicated that this was likely the main compiled Python bytecode file containing the ransomware logic.

Step 5: Identify the Two String Constants

Next, I examined the extracted Python bytecode file using strings.

```
┌──(kali㉿kali)-[~/Desktop/svchost_dump.exe_extracted]
└─$ strings -a svchost.pyc | less
```

```
encrypt_filer7
%USERPROFILE%\Documents)
*.txtz
*.docz
*.docxz
*.pdfr
DECRYPT
RANSOM
Software\ShadowCrypt\Info
        Encrypted
Method
XOR_MD5_YEAR
        Timestampa,
═══════════════════════════════════════════════════════════════════════
                    SHADOWCRYPT RANSOMWARE
═══════════════════════════════════════════════════════════════════════
YOUR FILES HAVE BEEN ENCRYPTED!
All your important documents are now locked with .lock extension.
Without our decryption key, recovery is IMPOSSIBLE.
───────────────────────────────────────────────────────────────────────
WHAT HAPPENED?
───────────────────────────────────────────────────────────────────────
Location: Documents folder
File Extension: .lock
───────────────────────────────────────────────────────────────────────
WARNING - TIME SENSITIVE!
───────────────────────────────────────────────────────────────────────
You have 72 HOURS to pay before your decryption key is deleted forever.
Price increases every 24 hours:
  0-24 hours:  0.5 BTC
  24-48 hours: 0.6 BTC
  48-72 hours: 0.7 BTC
  After 72h:   FILES LOST PERMANENTLY
═══════════════════════════════════════════════════════════════════════
                SHADOWCRYPT TEAM - Since 2019
           "Your files. Our encryption. Your choice."
═══════════════════════════════════════════════════════════════════════
```

Command used:

1. strings -a svchost.pyc | less

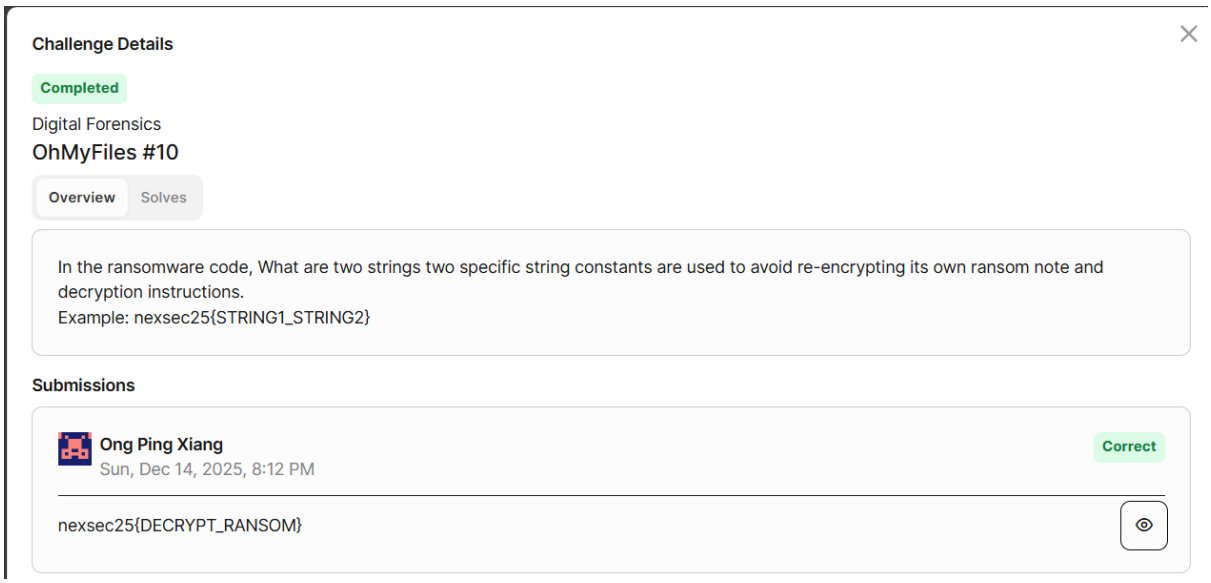
From the output, I found two important string constants:

- DECRYPT
- RANSOM

These two strings appeared to be used by the ransomware to recognize its own ransom note and decryption-related content, allowing it to avoid encrypting those files again.

Final Answer

The two string constants are "DECRYPT" and "RANSOM".



Flag**: nexsec25{DECRYPT_RANSOM}**